Grounding Large Language Models in Knowledge Graphs: A Comparative Review of Subgraph Retrieval, Agentic Planning, and Constrained Decoding

Carlos Gonzalez (uc2025195081) João Lino (uc2025246120)

1 Introduction and Course Context

Although large language models (LLMs) are able to produce fluent responses, their parametric knowledge can be incomplete, out of date, or inadequate when dealing with questions that require multiple relational steps. In the case of multi-hop question answering, an answer might be based on a series of entity–relation connections rather than on separate facts. The fact that the language used is plausible does not mean that these connections are supported by evidence. Knowledge graphs (KGs), by representing entities and relations as triples, provide a structured source against which reasoning can be grounded.

This challenge lies at the intersection of natural language processing and knowledge representation, central themes of the *Knowledge and Language* course. In particular, KG-augmented question answering combines neural language understanding and generation with graph-based relational modelling and structured reasoning. The key question is not simply whether an LLM can access a KG, but how graph structure actively shapes the process that produces an answer.

We look at three different approaches, which are distinguished by the time at which they incorporate the graph structure. *G-Retriever* (1) obtains a relevant connected subgraph and then uses both its textual and encoded forms for generation. *Plan-on-Graph* (PoG) (2) repeatedly explores paths in the knowledge graph by means of question decomposition, memory, and reflection. *Graph-Constrained* Reasoning (GCR) (3) limits the generated reasoning paths to those that are present in the knowledge graph. When these approaches are compared, it becomes clear that there are different trade-offs involving evidence selection, structural faithfulness, answer accuracy, and computational efficiency, which in turn leads to the research question formulated in this review.

2 Problem Settings, Assumptions, and Knowledge Sources

Task formulations. Although they vary in scope, all three methods use relational evidence to address natural-language queries. G-Retriever tackles general textual graph understanding, including knowledge-based question answering (QA), scene comprehension, and commonsense reasoning (1). PoG and GCR, on the other hand, concentrate on multi-hop knowledge graph QA (KGQA), where solutions must be deduced via connected entity–relation paths (2; 3).

Core assumptions. PoG assumes that topic entities mentioned in a question are already identified and linked to the knowledge graph (KG); GCR likewise starts from pre-identified question entities. Gold-answer links belong to benchmark annotations and training or evaluation, rather than being supplied as answers at inference. Consequently, entity linking is largely outside the evaluated graph-reasoning stage. Both methods depend on relevant facts being present and reachable, although neither guarantees KG completeness or factual correctness. GCR explicitly bounds candidate paths by a maximum hop count L, using two-hop paths in its main experiments. PoG instead explores iteratively under search and stopping constraints. G-Retriever's WebQSP setting uses two-hop Freebase subgraphs, but its broader textual graph tasks do not require the same entity-linking assumptions.

Knowledge sources and representations. G-Retriever evaluates on ExplaGraphs (commonsense relations), SceneGraphs (object attributes and spatial relations), and WebQSP (Freebase knowledge) (1). PoG uses Freebase-backed WebQSP, Complex WebQuestions (CWQ), and GrailQA (2). GCR evaluates primarily on WebQSP and CWQ, additionally testing transfer to FreebaseQA, ConceptNet-based CSQA, and MedQA using a medical KG derived from UMLS (3). While G-Retriever accommodates heterogeneous textual node and edge attributes, PoG and GCR primarily exploit explicit entity–relation–entity triples. SPARQL, used by PoG to query Freebase, is a querying mechanism rather than a knowledge representation format.

3 Methodological Synthesis: How Language Interacts with Graph Structure

Graph search and subgraph selection. The approaches differ fundamentally in how they identify graph evidence. G-Retriever employs a static, query-conditioned retrieval pipeline: Sentence-BERT embeds node and edge attributes, cosine similarity identifies the top-k candidates, and Prize-Collecting Steiner Tree (PCST) optimization extracts a compact connected subgraph, balancing semantic relevance against graph size (1). In contrast, PoG performs iterative exploration guided by an LLM. It decomposes questions into

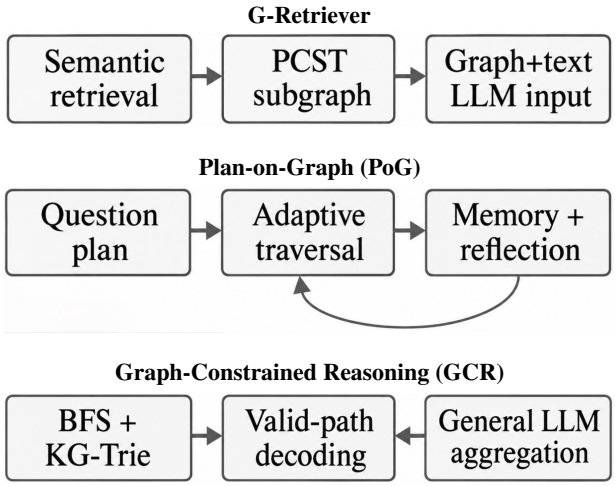


Figure 1: Comparison of graph-grounded reasoning pipelines: G-Retriever selects graph evidence before generation; PoG uses adaptive exploration and reflection; GCR constrains generated reasoning paths to valid KG paths. Structural validity does not guarantee answer correctness.

sub-objectives, adaptively selects relevant relations and entities, and retrieves graph neighborhoods through predefined SPARQL query templates (2). GCR instead uses breadth-first search (BFS) to enumerate paths within a maximum depth L from question entities. These paths are tokenized and compiled into a prefix-tree index (KG-Trie), enabling structurally constrained generation rather than repeated graph traversal (3). Thus, evidence is respectively selected before generation, discovered interactively, or encoded as decoding constraints (Figure 1).

Knowledge-to-LLM integration and reasoning. G-Retriever combines two representations of the selected subgraph: textualized node–edge information and a graph embedding projected into the LLM's input space as a learned soft-prompt token. Its main experiments use a Graph Transformer encoder, allowing structural and textual information to jointly condition generation (1). PoG instead incorporates retrieved facts directly into the LLM's reasoning context. Its Memory mechanism maintains explored subgraphs, reasoning paths, and sub-objective completion states, while Reflection evaluates whether to continue exploration or backtrack from unsuccessful paths (2). GCR introduces stronger structural control: a KG-specialized LLM generates candidate reasoning paths using beam search, with KG-Trie constraints masking tokens that would produce invalid graph paths. A general-purpose LLM subsequently aggregates candidate paths and hypothesis answers through inductive reasoning (3). Crucially, these hard constraints guarantee path membership in the KG, not the factual correctness of the final answer.

Adaptation and training regimes. The methods also impose different learning requirements. G-Retriever trains its graph encoder and projection layer while keeping the underlying LLM frozen, with optional Low-Rank Adaptation (LoRA) for additional fine-tuning (1). PoG is training-free, relying on prompting and the reasoning capabilities of existing LLMs, but requiring repeated model interactions (2). GCR fine-tunes its KG-specialized LLM using question–answer pairs and shortest supporting KG paths, while a separate general LLM performs answer aggregation. Its graph constraints can subsequently be replaced to support unseen KGs without additional model fine-tuning (3). Collectively, these designs illustrate different ways of combining neural language processing with explicit graph structure, trading retrieval simplicity, adaptive reasoning, and structural guarantees.

4 Evaluation Protocols, Empirical Results, and Efficiency Trade-offs

Evaluation protocols and metrics. The papers use partially overlapping benchmarks but different evaluation measures. G-Retriever reports accuracy for ExplaGraphs and SceneGraphs and Hit@1 for WebQSP, alongside a manual audit of 100 responses measuring valid cited nodes, edges, and complete supporting graphs (1). PoG reports Hits@1 on WebQSP, CWQ, and GrailQA (2). GCR instead reports Hit (whether any predicted answer is correct) and answer-set F1 on WebQSP and CWQ. Its faithfulness criterion checks whether generated reasoning paths exist in the KG (3). All three employ baselines and component ablations, but their metrics are not interchangeable.

Accuracy and faithfulness. On WebQSP, G-Retriever with LoRA achieves 73.79 Hit@1, compared with 66.03 for its LoRA-only baseline (1). PoG with GPT-4 reports 87.3 Hits@1, versus 82.6 for Think-on-Graph (ToG) (2; 4). GCR with Llama-3.1-8B and ChatGPT reports 92.6 Hit and 73.2 F1, compared with 90.7 Hit for GNN-RAG with retrieval augmentation (3; 5). However, these cross-paper scores do not establish a controlled ranking because models, graph settings, and answer metrics differ. Regarding faithfulness, G-Retriever increases fully valid graph references from 8% to 62% on SceneGraphs, whereas GCR enforces graph-valid generated paths by construction. Neither measure guarantees final-answer correctness, and PoG provides no directly comparable faithfulness score.

Computational efficiency and reasoning depth. G-Retriever reduces WebQSP graph-text tokens by approximately 99% and training time per epoch by 67%, without reporting comparable per-question latency (1). On WebQSP, PoG requires 9.0 LLM calls and 16.8 seconds per question, versus 15.9 calls and 63.1 seconds for ToG (2). GCR reports 2 LLM calls, 231 tokens, and 3.6 seconds on WebQSP (3). However, differing inference conditions and graph preprocessing costs prevent definitive cross-paper efficiency conclusions.

Component ablations. Within each paper's own WebQSP setting, removing retrieval reduces G-Retriever's Hit@1 from 70.49 to 63.84; removing reflection reduces PoG's Hits@1 from 82.0 to 78.1; and removing GCR's KG-specialized LLM reduces answer F1 from 73.2 to 52.9 (1; 2; 3). These are not comparable effect sizes, but they show that the three graph-integration mechanisms each contribute under their respective protocols.

5 Critical Limitations and Research Gap

Comparative failure modes. The three architectures exhibit distinct limitations. G-Retriever's static retrieval depends on initial semantic relevance scores; intermediate entities with weak query similarity may therefore be excluded, potentially disrupting multi-hop reasoning. The authors acknowledge static retrieval as a limitation (1). PoG addresses incorrect exploration through reflection and backtracking, but requires repeated LLM interactions and relies on the model's uncertain judgments about evidence sufficiency and when to stop (2). GCR guarantees that generated reasoning paths exist in the KG, but this does not ensure that they are relevant or factually correct, particularly when the graph contains missing or erroneous knowledge. Moreover, constructing its KG-Trie through breadth-first path enumeration has time complexity $O(b^L)$, where b is the average branching factor and L the maximum path length, creating scalability challenges for deeper reasoning (3).

Research gap and project motivation. Each approach leaves a complementary weakness: static retrieval leaves generation unconstrained, agentic exploration requires many LLM calls, and GCR's KG-Trie enumerates all L-hop paths around the question entities regardless of relevance. GCR's authors suggest building the KG-Trie on a question-related subgraph produced by a graph retriever to reduce this overhead, noting that performance may then be limited by retrieval accuracy (3); to our knowledge, this combination is not evaluated. This motivates a hybrid in which a PCST subgraph (1) is used to build a localized KG-Trie for graph-constrained decoding. Its central risk is that PCST may discard an intermediate entity on which the answer path depends (e.g., a parent in a two-hop kinship question), and constraining generation cannot recover a relation absent from the subgraph. A focused study should therefore compare LLM-only, RAG/RoG-style (6) and full-KG GCR-style baselines against the hybrid on the same questions and LLM, measuring answer accuracy, path validity, answer-path recall within the retrieved subgraph, and latency. Such a study connects neural language reasoning with explicit symbolic constraints while addressing a measurable question left open by the reviewed approaches.

References

[1] He, X.; Tian, Y.; Sun, Y.; Chawla, N. V.; Laurent, T.; LeCun, Y.; Bresson, X.; and Hooi, B. G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering. In *Advances in Neural Information Processing Systems*, 2024.

[2] Chen, L.; Tong, P.; Jin, Z.; Sun, Y.; Ye, J.; and Xiong, H. Plan-on-Graph: Self-Correcting Adaptive Planning of Large Language Model on Knowledge Graphs. In *Advances in Neural Information Processing Systems*, 2024.

[3] Luo, L.; Zhao, Z.; Haffari, G.; Li, Y.-F.; Gong, C.; and Pan, S. Graph-Constrained Reasoning: Faithful Reasoning on Knowledge Graphs with Large Language Models. In *Proceedings of the International Conference on Machine Learning*, 2025.

[4] Sun, J.; Xu, C.; Tang, L.; Wang, S.; Lin, C.; Gong, Y.; Ni, L.; Shum, H.-Y.; and Guo, J. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *International Conference on Learning Representations*, 2024.

[5] Mavromatis, C.; and Karypis, G. GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. In *Findings of the Association for Computational Linguistics: ACL*, 2025.

[6] Luo, L.; Li, Y.-F.; Haffari, G.; and Pan, S. Reasoning on Graphs: Faithful and Interpretable Large Language Model Reasoning. In *International Conference on Learning Representations*, 2024.